BTS SIO — Option SLAM

Session 2026

DOCUMENTATION TECHNIQUE

Application Web de Gestion

Ecole du Cinéma de Toulouse

Anaïs MARTINS
N° candidat : 02544722787
IRIS Mediaschool Toulouse

Épreuve E6 — Conception et développement d'applications (option SLAM)
Réalisation professionnelle n°1 — Client léger

Sommaire

1. Présentation du projet
2. Stack technique et environnement
3. Architecture du projet
4. Base de données
5. Authentification et sécurité
6. API REST
7. Interfaces utilisateur
8. Fonctionnalités transversales
9. Lancement et déploiement

1. Présentation du projet

1.1 Contexte

L'Ecole du Cinéma de Toulouse est un établissement proposant des formations aux métiers du cinéma : Réalisation, Acting et Scénarisation. Face au besoin de moderniser sa gestion administrative, l'école a souhaité se doter d'une application web centralisant l'ensemble de ses données : formations, apprenants, formateurs, cours, notes, présences et inscriptions.

Ce projet est réalisé dans le cadre du BTS SIO option SLAM, en tant que réalisation professionnelle n°1 (client léger). Il a été mené seule, de décembre 2025 à avril 2026, dans un environnement Windows 11 avec Visual Studio Code.

1.2 Objectifs

L'application répond à deux objectifs principaux :

- Proposer un site vitrine accessible à tous, présentant l'école, ses formations et ses coordonnées.
- Offrir un portail de gestion privé, accessible après connexion, dont les fonctionnalités varient selon le rôle de l'utilisateur.

1.3 Les trois profils utilisateurs

L'application est accessible à trois types d'utilisateurs, chacun disposant d'un espace personnalisé :

Profil	Accès et fonctionnalités
Administrateur	Accès complet : gestion des formateurs, apprenants, inscriptions, cours et comptes utilisateurs. Peut ajouter, modifier et supprimer chaque entité.
Formateur	Consultation de son planning de cours et de la liste de ses apprenants (avec coordonnées et avis déposés). Saisie, modification et suppression des notes.
Apprenant	Consultation de son emploi du temps, de ses notes, de ses présences et des coordonnées de ses formateurs. Possibilité de déposer un avis sur sa formation.

1.4 Comptes de test

Rôle	Identifiant	Mot de passe
Admin	anaismartins	mdp1
Formateur	merylstreep	mdp4
Apprenant	camilledupont	mdp2

2. Stack technique et environnement

L'application repose sur un ensemble d'outils et technologies complémentaires, chacun jouant un rôle précis dans le fonctionnement global.

2.1 Backend

Node.js (v23.3.0)

Node.js est un environnement d'exécution JavaScript côté serveur. Il permet d'exécuter du code JavaScript en dehors d'un navigateur, directement sur la machine. Dans ce projet, c'est lui qui fait tourner le serveur web : il reçoit les requêtes HTTP/HTTPS des navigateurs, les traite, interroge la base de données si nécessaire, et renvoie une réponse.

Express.js

Express est un framework web conçu pour Node.js. Il simplifie la création de serveurs web en fournissant un système de routage (définir quelle fonction répond à quelle URL), de middlewares (fonctions intermédiaires qui traitent la requête avant qu'elle n'atteigne la route), et de gestion des erreurs. Sans Express, il faudrait écrire tous ces mécanismes manuellement en Node.js pur.

Sequelize (ORM)

Sequelize est un ORM (Object-Relational Mapper). Un ORM est un outil qui fait le pont entre le code JavaScript et la base de données MySQL : plutôt que d'écrire des requêtes SQL à la main (SELECT * FROM apprenant WHERE id = 1), on manipule des objets JavaScript (Apprenant.findByPk(1)). Sequelize traduit automatiquement ces opérations en SQL. Il gère aussi les relations entre les tables (associations), la pagination, et la validation des données.

JWT — JSON Web Token (jsonwebtoken)

JWT est un standard d'authentification basé sur des jetons (tokens). Lors de la connexion, le serveur génère un token contenant les informations de l'utilisateur (identifiant, rôle, etc.) et le signe avec une clé secrète. Ce token est ensuite stocké dans un cookie du navigateur. À chaque requête suivante, le serveur vérifie la signature du token pour s'assurer de son authenticité, sans avoir besoin de consulter la base de données. C'est un système sans état (stateless) : le serveur ne conserve pas de session.

HTTPS avec certificat SSL auto-signé

HTTPS est la version sécurisée du protocole HTTP. Il chiffre toutes les communications entre le navigateur et le serveur, ce qui protège les données en transit (mots de passe, tokens, etc.). Un certificat SSL prouve l'identité du serveur. Ici, le certificat est dit auto-signé : il a été généré manuellement (fichiers key.pem et cert.pem) et n'est pas délivré par une autorité de certification officielle, ce qui convient pour un usage en développement local. Le navigateur affiche un avertissement de sécurité, mais la communication reste chiffrée.

2.2 Frontend

EJS (Embedded JavaScript)

EJS est un moteur de templates. Il permet de générer du HTML dynamiquement côté serveur, en y intégrant des variables JavaScript. Par exemple, la navbar affiche le nom de l'utilisateur connecté grâce à une variable injectée par le serveur. Les fichiers EJS ressemblent à du

HTML classique, avec en plus des balises spéciales (<%= variable %>, <%- include('partiel') %>, etc.) qui sont interprétées avant que la page soit envoyée au navigateur.

Bootstrap 5

Bootstrap est un framework CSS open-source. Il fournit un ensemble de classes CSS et de composants déjà stylisés (grilles, boutons, formulaires, tableaux...) qui permettent de construire rapidement des interfaces responsive. Dans ce projet, Bootstrap est utilisé principalement pour la structure des pages (grille, marges, alignements), tandis que le design spécifique à l'école est défini dans un fichier style.css personnalisé.

Fetch API (JavaScript natif)

La Fetch API est une interface native du navigateur qui permet de faire des requêtes HTTP asynchrones depuis le JavaScript côté client. Dans ce projet, elle est largement utilisée dans les dashboards pour interroger l'API REST sans recharger la page : chargement des cours, notes, présences, ajout ou suppression d'éléments, etc.

2.3 Base de données

MySQL

MySQL est un système de gestion de base de données relationnelle (SGBDR). Les données sont organisées en tables liées entre elles par des clés étrangères. Dans ce projet, la base s'appelle centre_formations et contient 13 tables. MySQL est accessible localement via XAMPP, et administré via phpMyAdmin.

phpMyAdmin

phpMyAdmin est une interface web permettant d'administrer une base de données MySQL sans écrire de commandes SQL dans un terminal. Il permet de visualiser les tables, exécuter des requêtes, importer ou exporter des données (fichier .sql). Il est accessible à l'adresse http://localhost/phpmyadmin.

2.4 Outils de développement

Visual Studio Code

Environnement de développement utilisé pour écrire l'ensemble du code. Extensions utilisées : coloration syntaxique EJS, ESLint (détection d'erreurs JavaScript), Prettier (formatage).

Postman

Postman est un outil permettant de tester des API REST. Il permet d'envoyer manuellement des requêtes HTTP (GET, POST, PUT, DELETE) vers les endpoints de l'API et d'inspecter les réponses JSON, sans avoir besoin d'une interface graphique. Utile pour valider le comportement de l'API avant d'intégrer le frontend.

XAMPP

XAMPP est un package logiciel regroupant Apache, MySQL et PHP. Dans ce projet, seul MySQL est utilisé. XAMPP lance et gère le serveur MySQL en local.

GitHub

Le code source est versionné et hébergé sur GitHub à l'adresse : github.com/Anais-Martins/Appli_Web-Ecole_du_cinema. Le dépôt contient également le fichier SQL de la base de données.

3. Architecture du projet

3.1 Vue d'ensemble

L'application suit une architecture MVC (Modèle - Vue - Contrôleur), adaptée au framework Express. Le modèle gère les données (Sequelize), la vue gère l'affichage (EJS), et le contrôleur gère la logique applicative (routes). Cette séparation des responsabilités rend le code plus lisible et plus maintenable.

Le flux d'une requête typique est le suivant :

- Le navigateur envoie une requête HTTPS vers le serveur (ex : GET /api/apprenant).
- Express reçoit la requête et la fait passer par les middlewares (Morgan pour les logs, cookieParser pour les cookies, setUser pour décoder le token JWT).
- La requête est transmise à la route correspondante (apiApprenant.js).
- La route appelle le modèle Sequelize (Apprenant.findAll()), qui traduit l'opération en SQL et interroge MySQL.
- Le résultat est renvoyé au navigateur sous forme de JSON (API) ou de page HTML rendue par EJS (interface).

3.2 Structure des dossiers

Voici la structure du projet et le rôle de chaque dossier :

config/

Contient la configuration de la connexion à la base de données. Le fichier db.js instancie Sequelize avec les paramètres de connexion MySQL (nom de la base, utilisateur, mot de passe, hôte, port). C'est ce fichier qui est importé dans tous les modèles.

models/

Chaque fichier dans ce dossier définit un modèle Sequelize, c'est-à-dire la représentation JavaScript d'une table de la base de données. Par exemple, apprenant.js définit les colonnes id_apprenant, prenom, nom, email, telephone, info_complementaires, avec leurs types et contraintes. Sequelize utilise ces définitions pour construire les requêtes SQL correspondantes.

routes/

C'est le cœur de la logique applicative. Ce dossier contient deux types de fichiers :

- Les routes API (préfixées api) : apiApprenant.js, apiFormateur.js, apiNote.js, etc. Elles reçoivent des requêtes, interagissent avec les modèles et renvoient du JSON.
- Les routes frontend : apprenant.js, formateur.js, etc. Elles rendent des vues EJS (.ejs) en réponse aux accès navigateur.
- Les routes d'authentification : auth.js (pages HTML de connexion/déconnexion) et apiAuth.js (API REST pour le traitement des formulaires).

views/

Contient tous les fichiers EJS qui constituent l'interface utilisateur. Organisé en sous-dossiers par entité (apprenant/, formateur/, cours/, etc.), chacun contenant trois vues : liste.ejs,

ajouter.ejs, modifier.ejs. Le sous-dossier partials/ contient les éléments réutilisés sur toutes les pages : navbar.ejs (barre de navigation) et footer.ejs (pied de page).

middlewares/

Contient les fonctions intermédiaires qui sont exécutées avant les routes. Le fichier setUser.js décode le token JWT présent dans les cookies et injecte les informations de l'utilisateur dans req.user (accessible dans les routes) et res.locals.user (accessible dans toutes les vues EJS). Le fichier auth.js protège les routes en vérifiant que l'utilisateur est bien connecté et possède le bon rôle.

public/

Dossier statique servi directement par Express. Contient le fichier CSS principal (public/stylesheets/style.css) ainsi que les images et polices utilisées dans le site vitrine.

app.js

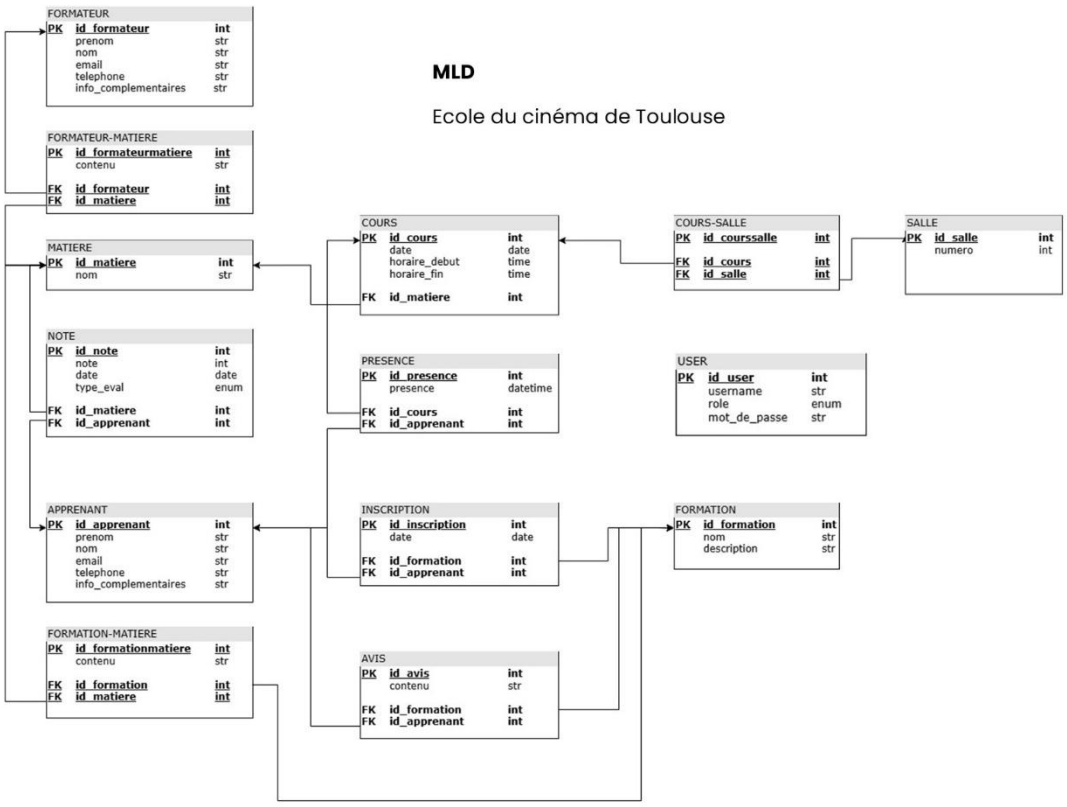
C'est le point d'entrée de l'application. Il instancie Express, déclare tous les middlewares, importe et monte toutes les routes, configure le moteur de vues EJS, et lance le serveur HTTPS sur le port 3001 après avoir vérifié la connexion à la base de données.

4. Base de données

4.1 Présentation générale

La base de données MySQL s'appelle centre_formations. Elle est composée de 13 tables qui modélisent l'ensemble des entités de l'école : personnes (apprenants, formateurs), contenus pédagogiques (formations, matières, cours), et données de suivi (notes, présences, inscriptions, avis). La clé primaire de chaque table est un entier auto-incrémenté. Les relations entre tables sont réalisées via des clés étrangères (FK).

4.2 Modèle Logique de Données (MLD)



4.3 Description des tables

formateur

Stocke les informations des formateurs de l'école. Champs : id_formateur (PK), prenom, nom, email, telephone, info_complementaires. Un formateur peut enseigner plusieurs matières (table de jonction formateur_matiere).

apprenant

Stocke les informations des apprenants. Champs : id_apprenant (PK), prenom, nom, email, telephone, info_complementaires. Un apprenant peut être inscrit à une formation (table inscription) et possède des notes, des présences et peut déposer un avis.

formation

Représente les trois formations proposées par l'école (Réalisation, Acting, Scénarisation). Champs : id_formation (PK), nom, description. Une formation est composée de plusieurs matières via la table formation_matiere.

matiere

Représente une discipline enseignée (ex : montage, scénario...). Champs : id_matiere (PK), nom. Sert de pivot entre les formations, les cours, les notes et les formateurs.

cours

Représente une séance de cours. Champs : id_cours (PK), date, horaire_debut, horaire_fin, id_matiere (FK). Un cours est associé à une matière et peut être rattaché à une salle via la table cours_salle.

presence

Enregistre la présence ou l'absence d'un apprenant à un cours. Champs : id_presence (PK), presence (booléen), id_cours (FK), id_apprenant (FK). La valeur true indique une présence, false une absence.

note

Stocke les notes attribuées aux apprenants. Champs : id_note (PK), note (valeur numérique), date, type_eval (ENUM : contrôle, examen, partiel...), id_matiere (FK), id_apprenant (FK). Les notes sont saisies par les formateurs via leur dashboard.

inscription

Table de jonction entre un apprenant et une formation. Champs : id_inscription (PK), date, id_formation (FK), id_apprenant (FK). Elle permet de savoir quels apprenants sont inscrits dans quelle formation.

avis

Stocke l'avis laissé par un apprenant sur sa formation. Champs : id_avis (PK), contenu (texte), id_formation (FK), id_apprenant (FK). Un apprenant ne peut déposer qu'un seul avis.

formation_matiere

Table de jonction entre formations et matières. Champs : id_formationmatiere (PK), id_formation (FK), id_matiere (FK). Elle permet de définir quelles matières composent chaque formation.

formateur_matiere

Table de jonction entre formateurs et matières. Champs : id_formateurmatiere (PK), id_formateur (FK), id_matiere (FK). Elle permet de savoir quel formateur enseigne quelle matière.

salle

Représente une salle de cours. Champs : id_salle (PK), numero. Liée aux cours via la table cours_salle.

user

Gère les comptes utilisateurs de l'application. Champs : id_user (PK), username, role (ENUM : apprenant / formateur / admin), mot_de_passe (hashé). Contient également id_apprenant pour lier un compte utilisateur à un apprenant. Le lien avec les formateurs est géré de manière applicative via la correspondance du nom d'utilisateur.

4.4 Sequelize et les modèles

Chaque table de la base de données correspond à un fichier modèle dans le dossier models/. Ces fichiers sont écrits en JavaScript et utilisés par Sequelize pour générer les requêtes SQL. Voici un exemple avec le modèle Apprenant :

```
const apprenant = sequelize.define('Apprenant', {
  id_apprenant: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey:
true },
  prenom:       { type: DataTypes.STRING, allowNull: false },
  nom:          { type: DataTypes.STRING, allowNull: false },
  email:        { type: DataTypes.STRING, allowNull: false },
  telephone:    { type: DataTypes.STRING, allowNull: false },
  info_complementaires: { type: DataTypes.STRING, allowNull: true }
}, { tableName: 'apprenant', timestamps: false });
```

Le paramètre tableName permet de spécifier le nom exact de la table MySQL. L'option timestamps: false désactive la création automatique des colonnes createdAt et updatedAt, qui ne sont pas présentes dans ce schéma.

5. Authentification et sécurité

5.1 HTTPS et certificat SSL auto-signé

Toutes les communications entre le navigateur et le serveur sont chiffrées via HTTPS. Le serveur Node.js utilise le module natif https (et non http) pour créer le serveur, en lui fournissant deux fichiers :

- key.pem : la clé privée du serveur, utilisée pour chiffrer les communications.
- cert.pem : le certificat auto-signé, qui identifie le serveur.

Ces fichiers sont générés manuellement (via OpenSSL) et placés à la racine du projet. Étant auto-signés (non délivrés par une autorité de certification reconnue), le navigateur affiche un avertissement « connexion non sécurisée », mais les données restent bien chiffrées. Cette approche est standard pour un développement en environnement local.

5.2 Authentification par JWT

Lorsque l'utilisateur soumet le formulaire de connexion, une requête POST est envoyée à l'endpoint /api/login. Le serveur vérifie les identifiants en base de données, puis, s'ils sont corrects, génère un token JWT contenant les informations suivantes :

- id_user : l'identifiant unique de l'utilisateur.
- username : le nom d'utilisateur.
- role : le rôle (apprenant, formateur, ou admin).
- id_apprenant : l'identifiant de l'apprenant lié (null si formateur ou admin).

Le token est ensuite stocké dans un cookie HTTP httpOnly. Cela signifie qu'il n'est pas accessible depuis JavaScript côté client, ce qui le protège contre les attaques XSS (Cross-Site Scripting). Le cookie est transmis automatiquement par le navigateur à chaque requête suivante.

5.3 Le middleware setUser

Le middleware setUser est exécuté avant toutes les routes, sur toutes les pages. Son rôle est de lire le token JWT depuis le cookie, de le vérifier et de le décoder. Si le token est valide, il injecte les informations de l'utilisateur dans :

- req.user : accessible dans les routes Express.
- res.locals.user : accessible directement dans toutes les vues EJS (utilisé notamment pour afficher le nom de l'utilisateur dans la navbar).

Si le token est absent, expiré ou invalide, ces valeurs sont mises à null et l'utilisateur est considéré comme non connecté.

5.4 Protection des routes

Un deuxième middleware, auth.js, protège les routes sensibles. Il vérifie que req.user existe et que son rôle correspond au rôle requis par la route. Par exemple, la route /dashboard/admin n'est accessible qu'aux utilisateurs avec le rôle admin. En cas de refus, l'utilisateur est redirigé vers la page de connexion.

5.5 Redirection selon le rôle

Après connexion, le frontend JavaScript lit le rôle contenu dans la réponse de l'API et redirige automatiquement l'utilisateur vers son dashboard :

- admin → /dashboard/admin
- formateur → /dashboard/formateur
- apprenant → /dashboard/apprenant

6. API REST

6.1 Qu'est-ce qu'une API REST ?

Une API (Application Programming Interface) est une interface qui permet à deux systèmes de communiquer. REST (Representational State Transfer) est un style d'architecture qui définit des règles pour concevoir des APIs web.

Dans ce projet, l'API REST est l'intermédiaire entre le frontend (les pages EJS et le JavaScript navigateur) et la base de données. Plutôt que de faire des requêtes SQL directement depuis le navigateur (impossible et dangereux), le frontend envoie des requêtes HTTP à l'API, qui interroge la base et renvoie les données au format JSON.

Une API REST utilise les verbes HTTP pour indiquer l'action souhaitée :

Verbe HTTP	Action
GET	Lire / consulter des données
POST	Créer une nouvelle entrée
PUT	Modifier une entrée existante
DELETE	Supprimer une entrée

6.2 Convention de nommage

Dans ce projet, toutes les routes API commencent par /api/ et utilisent le nom de la ressource au singulier. Les routes sont montées dans app.js et chaque ressource dispose de son propre fichier de routes dans le dossier routes/.

> *Exemple : toutes les opérations sur les apprenants sont gérées par /api/apprenant, défini dans routes/apiApprenant.js et monté via app.use('/api/apprenant', apiApprenantRouter).*

6.3 Endpoints disponibles

/api/apprenant

Méthode + Route	Description
GET /api/apprenant	Liste paginée avec recherche (param. page, limit, search)
GET /api/apprenant/:id	Détail d'un apprenant
POST /api/apprenant	Créer un apprenant
PUT /api/apprenant/:id	Modifier un apprenant
DELETE /api/apprenant/:id	Supprimer un apprenant
GET /api/apprenant/:id/cours	Cours d'un apprenant (via inscriptions)
GET /api/apprenant/:id/presences	Présences d'un apprenant
GET /api/apprenant/:id/notes	Notes d'un apprenant

GET /api/apprenant/:id/formateurs	Formateurs d'un apprenant
GET /api/apprenant/:id/avis	Avis déposé par un apprenant
GET /api/apprenant/:id/formations	Formations d'un apprenant

/api/formateur

Méthode + Route	Description
GET /api/formateur	Liste de tous les formateurs
GET /api/formateur/:id	Détail d'un formateur
POST /api/formateur	Créer un formateur
PUT /api/formateur/:id	Modifier un formateur
DELETE /api/formateur/:id	Supprimer un formateur
GET /api/formateur/:id/cours	Planning des cours d'un formateur
GET /api/formateur/:id/apprenants	Apprenants suivis par un formateur

Autres ressources

Route de base	Ressource gérée
/api/formation	Formations (CRUD)
/api/matiere	Matières (CRUD)
/api/cours	Cours (CRUD)
/api/note	Notes (CRUD)
/api/presence	Présences (CRUD)
/api/inscription	Inscriptions (CRUD)
/api/avis	Avis (CRUD)
/api/user	Comptes utilisateurs (CRUD)
/api/salle	Salles (CRUD)
/api/formateur_matiere	Associations formateur-matière
/api/formation_matiere	Associations formation-matière
/api/login	Authentification (POST)
/api/logout	Déconnexion (POST)

6.4 Exemple de fonctionnement : récupération des apprenants

Voici le déroulement complet d'une requête GET /api/apprenant?page=1&limit=10&search=dupont :

- Le navigateur envoie la requête via fetch().
- Express achemine la requête vers le fichier routes/apiApprenant.js.

- Le handler lit les paramètres page, limit et search depuis req.query.
- Il construit une condition Sequelize avec Op.or et Op.like pour filtrer sur nom, prenom et email.
- Sequelize exécute une requête SQL de type SELECT ... WHERE nom LIKE '%dupont%' ... LIMIT 10 OFFSET 0.
- Le résultat (liste d'apprenants + métadonnées de pagination) est renvoyé au format JSON.

Format de la réponse JSON :

```
{ "apprenants": [ { "id_apprenant": 1, "prenom": "Camille", "nom": "Dupont",
... } ],
  "pagination": { "page": 1, "totalPages": 3, "totalItems": 25 } }
```

6.5 Gestion des erreurs

Chaque route est encapsulée dans un bloc try/catch. En cas d'erreur, la route renvoie un objet JSON avec le message d'erreur et le code HTTP approprié :

- 400 Bad Request : données invalides (ex : ID non numérique).
- 404 Not Found : ressource introuvable.
- 500 Internal Server Error : erreur inattendue côté serveur.

Un middleware global de gestion des erreurs est défini à la fin de app.js pour intercepter toutes les erreurs non gérées.

7. Interfaces utilisateur

7.1 Charte graphique

L'ensemble du site adopte une identité visuelle sobre et élégante, cohérente avec l'univers du cinéma. Elle repose sur trois couleurs principales et deux familles typographiques :

Elément	Valeur
Couleur principale (noir)	#0d0d0d
Couleur d'accent (beige)	#FCE4C2
Fond général	#FFFFFF
Police des titres	Cormorant Garamond (serif élégant)
Police du corps	Jost (sans-serif lisible)

La bannière en haut de chaque page est fond noir avec une bordure basse beige, et affiche le titre « Ecole du Cinéma de Toulouse » en italique beige. La navbar est fond blanc. Il n'y a pas de border-radius ni de couleurs vives : le design est minimaliste et élégant.

7.2 Site vitrine

Le site vitrine est accessible à tous, sans connexion. Il comprend quatre pages :

- Accueil (/) : page héro avec une photo de projecteur de cinéma en fond, un titre et un slogan, et les liens de navigation.
- Formations (/formations) : présentation des trois formations (Réalisation, Acting, Scénarisation) sous forme d'encarts.
- Contact (/contact) : informations de contact de l'école.
- Connexion (/connexion) : formulaire de connexion avec un champ identifiant et un champ mot de passe.

Non sécurisé https://localhost:3001/formations

Ecole du Cinéma de Toulouse

Accueil Formations Contact Connexion

Nos formations

Réalisation	Cours & formateurs	
Formation complète aux métiers de la réalisation cinématographique. Maîtrise de la direction d'acteurs, du cadrage, de la lumière, du montage narratif et de la préparation technique d'un tournage.	Cadrage et composition	*Wes Anderson*
	Direction d'acteurs	*Xavier Dolan*
	Montage narratif	*Lana Wachowski*
	Lumière et atmosphère	*Greta Gerwig*
	Préparation et découpage technique	*Alfred Hitchcock*

Acting	Cours & formateurs	
Formation intensive au jeu d'acteur pour la caméra et la scène. Travail approfondi sur les techniques d'interprétation, la gestion des émotions, la préparation physique et vocale du personnage, et l'improvisation.	Techniques d'improvisation	*Meryl Streep*
	Exploration des émotions	*Florence Pugh*
	Préparation physique du personnage	*Marina Foïs*
	Travail vocal et diction	*Viola Davis*
	Jeu face caméra	*Leïla Bekhti*

Non sécurisé https://localhost:3001/contact

Ecole du Cinéma de Toulouse

Accueil Formations Contact Connexion

Nous contacter

Coordonnées

ADRESSE
9 Rue des Amidonniers, 31000 Toulouse

QUARTIER
Proche Place Saint-Pierre

TÉLÉPHONE
05 06 07 08 09

Mentions légales Politique de confidentialité CGU

© 2026 – Ecole du Cinéma de Toulouse – Tous droits réservés

7.3 Dashboard Administrateur

Après connexion avec un compte admin, l'utilisateur accède à /dashboard/admin. Cette page présente une grille de cinq encarts, chacun correspondant à une entité gérable :

- Formateurs → /formateur
- Apprenants → /apprenant
- Inscriptions → /inscription
- Cours → /cours
- Utilisateurs → /register

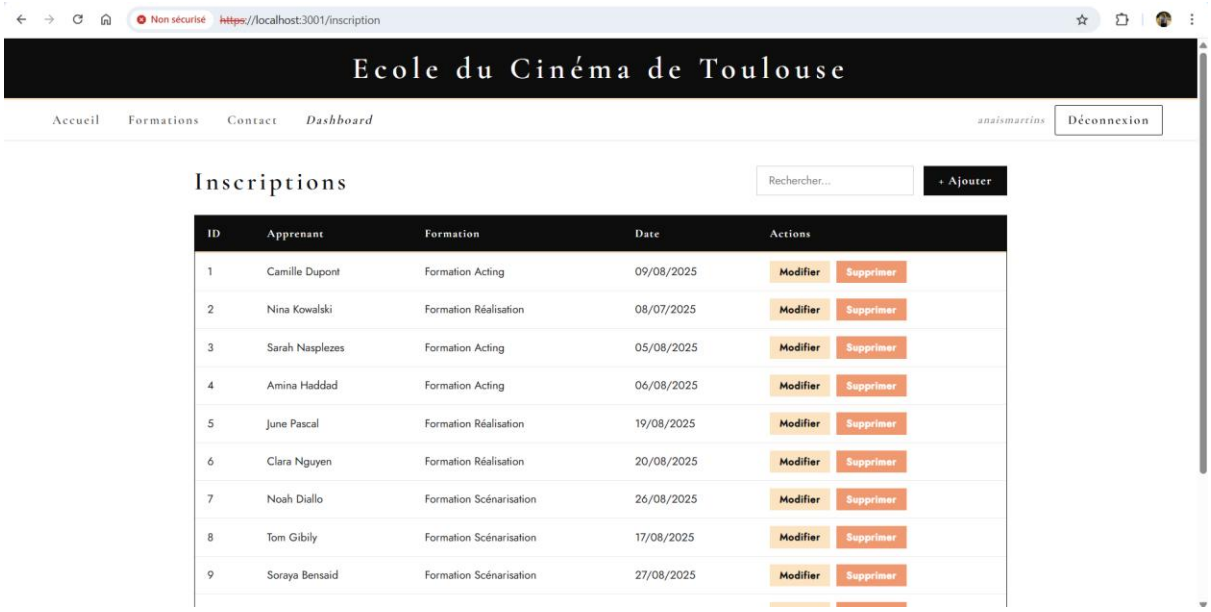
Pages CRUD

Chaque entité dispose de trois pages :

- Liste : tableau avec toutes les entrées, barre de recherche en temps réel, pagination, et boutons Modifier / Supprimer sur chaque ligne. La suppression est effectuée via un appel DELETE à l'API REST, sans rechargement de la page.
- Ajouter : formulaire de saisie avec validation. La soumission envoie un POST à l'API REST. Un message de succès ou d'erreur s'affiche directement sous le formulaire.
- Modifier : identique au formulaire d'ajout, mais prérempli avec les données existantes. La soumission envoie un PUT à l'API REST.

Ecole du Cinéma de Toulouse
Accueil
Formations
Contact
Dashboard
anaismartins
Déconnexion
Modifier un formateur
PRÉNOM
Meryl
NOM
Streep
E-MAIL
meryl.sl@gmail.com
TÉLÉPHONE
555789
INFORMATIONS COMPLÉMENTAIRES
← Retour
Enregistrer

Non sécurisé
https://localhost:3001/apprenant
Ecole du Cinéma de Toulouse
Accueil
Formations
Contact
Dashboard
anaismartins
Déconnexion
Apprenants
Rechercher...
+ Ajouter
ID Prénom Nom E-mail Téléphone Formation(s) Actions
1 Camille Dupont camille.dupont@gmail.com 0601020304 Formation Acting Modifier Supprimer
2 Nina Kowalski nina.kowalski@gmail.com 0601020305 Formation Réalisation Modifier Supprimer
3 Noah Diallo noahdiallo@outlook.com 0601020345 Formation Scénarisation Modifier Supprimer
4 Sarah Nasplezes snasplezes@outlook.com 0601020308 Formation Acting Modifier Supprimer
5 Amina Haddad amina_haddad@yahoo.fr 0601020309 Formation Acting Modifier Supprimer
6 June Pascal june_pascal@icloud.com 0601020318 Formation Réalisation Modifier Supprimer
7 Clara Nguyen clara.nguyen@protonmail.com 0601020319 Formation Réalisation Modifier Supprimer
8 Tom Gibily tom.gibily@outlook.com 0601020328 Formation Scénarisation Modifier Supprimer
9 Soraya Bensaid soraya.bensaid@icloud.com 0601020329 Formation Scénarisation Modifier Supprimer


Non sécurisé
https://localhost:3001/inscription
Ecole du Cinéma de Toulouse
Accueil
Formations
Contact
Dashboard
anaismartins
Déconnexion
Inscriptions
Rechercher...
+ Ajouter
ID
Apprenant
Formation
Date
Actions
1 Camille Dupont Formation Acting 09/08/2025 Modifier Supprimer
2 Nina Kowalski Formation Réalisation 08/07/2025 Modifier Supprimer
3 Sarah Nasplezes Formation Acting 05/08/2025 Modifier Supprimer
4 Amina Haddad Formation Acting 06/08/2025 Modifier Supprimer
5 June Pascal Formation Réalisation 19/08/2025 Modifier Supprimer
6 Clara Nguyen Formation Réalisation 20/08/2025 Modifier Supprimer
7 Noah Diallo Formation Scénarisation 26/08/2025 Modifier Supprimer
8 Tom Gibily Formation Scénarisation 17/08/2025 Modifier Supprimer
9 Soraya Bensaid Formation Scénarisation 27/08/2025 Modifier Supprimer

Ecole du Cinéma de Toulouse
Accueil
Formations
Contact
Dashboard
anaismartins
Déconnexion
Cours
Rechercher...
+ Ajouter
ID
Matière
Date
Début
Fin
Actions
1 Lumière et atmosphère 04/09/2025 09:00 12:00 Modifier Supprimer
2 Cadrage et composition 08/09/2025 10:00 13:00 Modifier Supprimer
3 Technique d'improvisation 09/09/2025 16:00 18:00 Modifier Supprimer
4 Direction d'acteurs 18/09/2025 14:00 17:00 Modifier Supprimer
5 Préparation physique du personnage 22/09/2025 16:00 18:00 Modifier Supprimer
6 Préparation et découpage technique 24/09/2025 09:00 12:00 Modifier Supprimer
7 Exploration des émotions 26/09/2025 10:00 13:00 Modifier Supprimer
8 Jeu face caméra 29/09/2025 14:00 16:00 Modifier Supprimer
9 Adaptation littéraire 30/09/2025 16:00 18:00 Modifier Supprimer

7.4 Dashboard Formateur

Après connexion avec un compte formateur, l'utilisateur accède à /dashboard/formateur. L'interface est organisée en trois onglets, chargés dynamiquement via des appels à l'API REST :

- Mes cours : tableau listant les séances du formateur (matière, date, horaire de début et de fin). Les données sont récupérées via GET /api/formateur/:id/cours.
- Mes apprenants : tableau listant les apprenants du formateur avec leurs coordonnées (prénom, nom, e-mail, formation) et l'avis qu'ils ont laissé sur leur formation.
- Notes : tableau des notes saisies, avec la possibilité d'en ajouter, modifier et supprimer directement depuis l'interface via des formulaires inline.

Ecole du Cinéma de Toulouse

Accueil Formations Contact *Dashboard* *merylstreep* Déconnexion

Mon espace formateur

Mes cours | **Mes apprenants** | Notes

Prénom	Nom	E-mail	Formation	Avis sur la formation
Camille	Dupont	camille.dupont@gmail.com	Formation Acting	Formation très complète et profs au top !
Sarah	Nasplezes	snasplezes@outlook.com	Formation Acting	Très bonne formation d'acting, le jeu face caméra et l'impro m'ont fait gagner en confiance.
Amina	Haddad	amina_haddad@yahoo.fr	Formation Acting	*Pas d'avis*
Chloé	Moreau	c.moreau@icloud.com	Formation Acting	*Pas d'avis*
Loïc	Dubois	loicdubois@yahoo.fr	Formation Acting	*Pas d'avis*
Patrick	Mensah	p.mensah@outlook.com	Formation Acting	*Pas d'avis*

Ecole du Cinéma de Toulouse

Accueil Formations Contact *Dashboard* *merylstreep* Déconnexion

Mon espace formateur

Mes cours | Mes apprenants | **Notes**

+ Ajouter une note

Apprenant	Matière	Note	Date	Type d'évaluation	Actions
Alexei Ivanov	Technique d'improvisation	11/20	23/12/2025	eval_simple	Modifier Supprimer
Patrick Mensah	Technique d'improvisation	19/20	23/12/2025	eval_simple	Modifier Supprimer
David Léva	Technique d'improvisation	8/20	22/12/2025	DM	Modifier Supprimer
Alexei Ivanov	Technique d'improvisation	14/20	18/12/2025	oral	Modifier Supprimer
Camille Dupont	Technique d'improvisation	12/20	18/12/2025	partiel	Modifier Supprimer
Loïc Dubois	Technique d'improvisation	16/20	15/12/2025	DM	Modifier Supprimer
Amina Haddad	Technique d'improvisation	11/20	12/12/2025	DM	Modifier Supprimer

Non sécurisé ~~https~~://localhost:3001/dashboard/formateur

Ecole du Cinéma de Toulouse

Accueil Formations Contact *Dashboard* *merylstreep* Déconnexion

Ajouter une note

APPRENANT

– Sélectionner un apprenant –

MATIÈRE

– Sélectionner une matière –

NOTE (SUR 20)

ex: 14

DATE

jj/mm/aaaa

TYPE D'ÉVALUATION

– Sélectionner –

Enregistrer Annuler

7.5 Dashboard Apprenant

Après connexion avec un compte apprenant, l'utilisateur accède à /dashboard/apprenant. L'interface est organisée en quatre onglets :

- Cours : emploi du temps de l'apprenant avec, pour chaque séance, le statut de présence (badge vert « Présent », badge rouge « Absent », ou badge gris « À venir » pour les cours futurs). Les cours et les présences sont chargés en parallèle via Promise.all().
- Notes : tableau listant toutes les notes de l'apprenant, avec la matière, la valeur, la date et le type d'évaluation.
- Formateurs : liste des formateurs de l'apprenant avec leurs coordonnées et la matière enseignée.
- Avis : si l'apprenant a déjà déposé un avis, il est affiché avec des options de modification et de suppression. Sinon, un formulaire permet d'en créer un.

Non sécurisé https://localhost:3001/dashboard/apprenant

Ecole du Cinéma de Toulouse

Accueil Formations Contact *Dashboard* *camilledupont* Déconnexion

Mon espace

Cours Notes Formateurs Avis

Matière	Date	Début	Fin	Présence
Technique d'improvisation	09/09/2025	16:00	18:00	Présent
Préparation physique du personnage	22/09/2025	16:00	18:00	Présent
Exploration des émotions	26/09/2025	10:00	13:00	Présent
Jeu face caméra	29/09/2025	14:00	16:00	Présent
Travail vocal et diction	08/10/2025	16:00	19:00	Absent
Technique d'improvisation	15/10/2025	09:00	12:00	Présent
Exploration des émotions	16/10/2025	10:00	13:00	Présent
Préparation physique du personnage	17/10/2025	14:00	17:00	Présent
Travail vocal et diction	22/10/2025	10:00	13:00	Présent

Non sécurisé https://localhost:3001/dashboard/apprenant

Ecole du Cinéma de Toulouse

Accueil Formations Contact *Dashboard* *camilledupont* Déconnexion

Mon espace

Cours Notes Formateurs Avis

Matière	Note	Date	Type d'évaluation
Préparation physique du personnage	14	17/09/2025	eval_simple
Travail vocal et diction	16	22/10/2025	partiel
Technique d'improvisation	11	27/10/2025	eval_simple
Technique d'improvisation	17	30/10/2025	DM
Jeu face caméra	16	31/10/2025	partiel
Travail vocal et diction	14	03/11/2025	oral
Travail vocal et diction	8	07/11/2025	eval_simple
Travail vocal et diction	15	12/11/2025	partiel
Préparation physique du personnage	17	13/11/2025	eval_simple

8. Fonctionnalités transversales

8.1 Partials EJS : navbar et footer

La navbar et le footer sont définis une seule fois dans views/partials/navbar.ejs et views/partials/footer.ejs, et inclus dans toutes les pages grâce à la directive EJS <%- include('partials/navbar') %>. La navbar est dynamique : elle affiche le lien « Dashboard » et le bouton de déconnexion uniquement si un utilisateur est connecté (vérification via la variable user injectée par setUser).

8.2 Recherche en temps réel

Les pages liste (formateurs, apprenants, cours...) disposent d'un champ de recherche qui filtre les résultats au fur et à mesure de la frappe, sans recharger la page. Cela fonctionne via un

événement oninput qui déclenche un appel fetch vers l'API avec le paramètre search. Côté serveur, Sequelize utilise l'opérateur Op.like de SQL pour filtrer les résultats sur plusieurs colonnes (nom, prénom, email...).

8.3 Pagination

Les listes longues sont paginées : chaque page affiche un nombre limité d'entrées (10 par défaut). L'API reçoit les paramètres page et limit, calcule l'offset (décalage dans la liste), et retourne avec les données le nombre total de pages. Le frontend génère dynamiquement les boutons de navigation.

8.4 Messages de succès et d'erreur

Les formulaires (ajout, modification) affichent un message visuel sous le formulaire après soumission, sans rechargement de page. Un message vert (classe CSS .formulaire-succes) confirme le succès de l'opération. Un message rouge (classe .formulaire-erreur) signale une erreur renvoyée par l'API.

8.5 Classes CSS réutilisables

L'ensemble du design est centralisé dans public/stylesheets/style.css. Des classes réutilisables ont été définies pour assurer la cohérence visuelle :

- .liste-section, .liste-table, .liste-wrapper, .liste-header, .liste-search : structure des pages liste.
- .formulaire-card, .formulaire-card-header, .formulaire-card-body, .formulaire-champ : structure des formulaires.
- .dashboard-card, .dashboard-card-header, .dashboard-card-body : encarts du dashboard admin.
- .btn-onglet, .btn-onglet.actif, .onglet-content : système d'onglets des dashboards.
- .badge-present, .badge-absent, .badge-futur : badges colorés pour les présences.

9. Lancement et déploiement

9.1 Prérequis

- Node.js v23.3.0 installé
- XAMPP lancé avec le module MySQL actif
- Base de données centre_formations importée dans phpMyAdmin (à partir du fichier centre_formations.sql inclus dans le dépôt)
- Fichiers key.pem et cert.pem présents à la racine du projet

9.2 Installation et lancement

Cloner le dépôt ou dézipper l'archive, puis installer les dépendances :

```
npm install
```

Lancer le serveur en mode développement :

```
npm run dev
```

L'application est alors accessible à l'adresse : https://localhost:3001

En cas d'avertissement de sécurité du navigateur (lié au certificat auto-signé), cliquer sur « Paramètres avancés » puis « Continuer vers le site ».

9.3 Accès aux outils

- phpMyAdmin : http://localhost/phpmyadmin
- Code source : https://github.com/Anais-Martins/Appli_Web-Ecole_du_cinema
- Documentation en ligne : https://anais-martins.github.io/projet-situation-pro-1.html